Aegis — An Encrypted "Dead Man's Switch" Legacy Vault

Software Requirements Specification (SRS) for UCS503P
Submitted to: Dr. Paramveer Sidhu
Thapar Institute of Engineering and Technology

Pratham Arora, Krishna Pandey, Nipun Behl (COE)

August 17, 2026

Abstract

Aegis is an encrypted "dead man's switch" legacy vault. An *Owner* deposits encrypted files or messages into a *vault* and designates N *trustees* with a reconstruction *threshold* K. The system periodically requires the Owner to *check in*; if a check-in is missed and a configured *grace period* lapses, the decryption key — pre-split using Shamir's Secret Sharing so that any K of N shares reconstruct it and any $K-1$ reveal nothing — is distributed to the trustees, who together reconstruct the key and open the vault. This document specifies the software requirements for version 1 (v1) of Aegis.

1 Problem Statement

. . . the gap this project addresses

Individuals routinely hold digital secrets that they cannot safely disclose while alive, yet which must not be lost when they die or become permanently incapacitated: banking and brokerage credentials, cryptocurrency private keys, password-manager master passwords, encrypted archives, business continuity information, and personal messages intended for specific recipients. The value of such data is entirely conditional on *access*, and access is precisely what is destroyed by the owner's absence.

Existing practice offers two options, and both are unsatisfactory:

(a) **Disclose now.** The owner shares the secret with a trusted person or writes it into a document held by a custodian. This converts a confidentiality problem into a trust problem that must hold indefinitely: the secret is readable from the moment it is shared, and its safety depends on the continued honesty, competence, and availability of a single party.

(b) **Disclose never.** The owner tells no one. Confidentiality is preserved perfectly, and the data is irrecoverably lost on the owner's death.

Conventional instruments such as sealed wills and escrow arrangements sit within option (a) and inherit its central weakness — a **single point of failure**. One document, one custodian, and one occasion on which it is opened: if the document is destroyed, the custodian is compromised or unreachable, or the release is contested, the owner's intent is not carried out. Furthermore, these instruments are triggered by an external legal or administrative process rather than by the absence of the owner, and are therefore unsuitable for digital assets that require timely release.

The problem this project addresses is therefore:

How can a system release a secret to designated recipients when, and only when, its owner has become unresponsive — without any single party, including the system operator, being able to read that secret beforehand, and without the release depending on any single custodian?

A solution must satisfy three properties simultaneously, and it is their combination that makes the problem non-trivial:

- **Conditional release.** Disclosure must be triggered automatically by the owner's sustained non-response, not by a human decision.
- **Distributed trust.** No single recipient may hold enough material to open the vault alone, so that neither the loss nor the betrayal of one party is sufficient to defeat the system.
- **Temporal safety.** The system must never disclose early. A premature release is unrecoverable — the secret cannot be un-revealed — and is therefore treated in this specification as the catastrophic failure mode (`NFR-REL-1`).

Aegis addresses this problem by combining a *dead man's switch* — a periodic check-in whose absence, rather than whose presence, is the trigger — with *Shamir's Secret Sharing*, which splits the vault's decryption key into N shares such that any K reconstruct it and any $K-1$ reveal nothing about it in an information-theoretic sense. The former provides conditional release; the latter eliminates the single custodian.

2 Purpose

. . . why this document exists

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of **Aegis**, an encrypted legacy vault built around a time-triggered release mechanism. It is the authoritative reference for the design, implementation, and evaluation of v1 across the UCS503P project timeline. The intended readers are the lab instructor/evaluator (as the acceptance authority) and the project team (as the developers). Each functional requirement carries a traceable identifier (`FR-`n) and each non-functional requirement a category identifier (`NFR-`cat`-`n) so that later design, code, and tests can cite the requirement they satisfy.

3 Scope

. . . what v1 is, and is not

3.1 In scope (v1)

Aegis v1 is a web application with a Python/FastAPI backend and a React frontend. It provides:

- account registration and authentication for Owners;
- creation of a single vault per Owner and upload of an encrypted payload;
- configuration of a check-in interval and a grace period;
- designation of N trustees and a threshold K ($1 \le K \le N$);
- a scheduler that issues check-in prompts and, on missed check-in past the grace period, distributes key shares to trustees;
- a one-action check-in confirmation for the Owner;
- trustee-side submission of K shares to reconstruct the key and decrypt the payload.

The algorithmic core is a hand-written implementation of *Shamir's Secret Sharing* (SSS) over a finite field, together with symmetric encryption of the payload under the shared key.

3.2 Non-goals *... explicit exclusions for v1*

The following are **explicitly out of scope** for v1 and must not be assumed by design or evaluation:

NG-1. No SMS. All notifications are delivered by email (SMTP) only; no SMS or telephony channel is provided.

NG-2. No mobile app. Aegis v1 is a responsive web application only; no native Android/iOS client is built.

NG-3. Single vault per user. Each Owner has at most one vault in v1; multi-vault management is deferred.

4 Glossary *... shared vocabulary*

Term	Definition
Vault	The encrypted container owned by a single Owner, holding one payload plus its release configuration and lifecycle state.
Payload	The Owner's secret data (files and/or messages), stored only as ciphertext.
Share	One piece of the split decryption key produced by Shamir's Secret Sharing; held by a single trustee.
Threshold (K)	The minimum number of shares required to reconstruct the key; any K of N suffice, any $K-1$ reveal nothing.
Grace period	The additional time granted after a missed check-in before the vault is released.
Check-in	An explicit, confirmed action by which the Owner signals they are still present, resetting the release timer.
Owner	The registered user who creates a vault, uploads the payload, and is prompted to check in.
Trustee	A person designated by the Owner to receive a key share and participate in reconstruction after release.
Scheduler	The time-triggered system actor that tracks deadlines and drives the vault through its lifecycle.

5 System actors and models *... see `docs/` diagrams*

Aegis recognises three actors: the **Owner** and **Trustee** (human) and the **Scheduler** (a time-triggered system actor that initiates prompts and release without human action). The following models accompany this SRS and are maintained as Mermaid diagrams in the project documentation:

- **Use-case diagram** — `docs/diagrams/use-case.md` (actors Owner, Trustee, Scheduler).
- **Vault lifecycle state diagram** — `docs/diagrams/vault-lifecycle.md` (Active → Warning → Grace → Released).
- **High-level architecture** — `docs/diagrams/architecture.md`.

6 Functional Requirements

. . . traceable `FR-n`

`FR-1` **Registration & authentication.**
The system shall allow a visitor to register as an Owner with an email and password, and shall authenticate returning Owners. Credentials shall be verified before any vault operation. Passwords are never stored in plaintext (see `NFR-SEC-3`).

`FR-2` **Vault creation & payload upload.**
An authenticated Owner shall be able to create their (single) vault and upload a payload (files and/or a text message). The payload shall be encrypted client- or server-side and persisted only as ciphertext (see `NFR-SEC-1`).

`FR-3` **Timing configuration.**
The Owner shall configure the *check-in interval* (how often they must check in) and the *grace period* (additional time after a missed check-in before release). Both are positive durations; the grace period may be zero only if explicitly chosen.

`FR-4` **Trustee designation & threshold.**
The Owner shall designate $N \geq 1$ trustees (by email) and set a threshold K with $1 \leq K \leq N$. On confirmation, the system shall split the vault key into N Shamir shares with threshold K.

`FR-5` **Scheduled check-in prompts.**
The Scheduler shall, at each interval boundary, transition the vault to *Warning* and prompt the Owner to check in via email (see `NFR-PERF-2`).

`FR-6` **One-action check-in confirmation.**
The Owner shall be able to confirm a check-in with a single action (one click on a tokenised link or button), which resets the release timer and returns the vault to *Active*.

`FR-7` **Share distribution on expiry.**
If no valid check-in is confirmed before the interval deadline *and* the grace period have both elapsed, the Scheduler shall transition the vault to *Released* and distribute one share to each trustee by email — and shall do so **only then** (see `NFR-REL-1`).

`FR-8` **K-of-N reconstruction & decryption.**
After release, the system shall accept shares submitted by trustees and, upon receiving any K valid shares, reconstruct the key and decrypt the payload for retrieval. Fewer than K shares shall not permit decryption.

7 Non-Functional Requirements

. . . measurable `NFR-cat-n`

7.1 Reliability

. . . the catastrophic failure mode

`NFR-REL-1` **No early release.**
The system shall **never** release a vault (transition to *Released* or distribute any share) before the check-in deadline *and* the full grace period have elapsed with no confirmed check-in. *Measure:* across an automated reliability suite of ≥ 1000 simulated schedules — including process restarts, timer resets, retries, and system-clock adjustments — the count of releases occurring earlier than `deadline + grace` shall be exactly **zero**. This is the project's single most important requirement.

`NFR-REL-2` **Durable deadlines.**
Pending release deadlines shall survive a process/host restart. *Measure:* after a forced restart at an arbitrary point, every previously scheduled deadline is re-armed and fires no later than 60 s after its due time, with no deadline lost and none fired early.

`NFR-REL-3` **Exactly-once release.**
Each vault shall be released at most once and each trustee shall receive at most one share. *Measure:* no duplicate share emails under retry/concurrent execution in the reliability suite.

7.2 Security *. . . payload confidentiality*

`NFR-SEC-1` **Payload never in plaintext.**
The payload shall never be persisted in plaintext; only ciphertext plus non-secret metadata (filename, size, MIME type) may be stored. *Measure:* an automated inspection of the database and file store finds no plaintext payload for any vault; the plaintext key exists only transiently in memory during encryption and reconstruction.

`NFR-SEC-2` **Threshold secrecy.**
Any set of at most $K-1$ shares shall reveal no information about the key. *Measure:* property tests confirm that all $\binom{N}{K-1}$ share subsets are statistically consistent with every possible secret (an intrinsic guarantee of SSS).

`NFR-SEC-3` **Credential protection.**
Passwords shall be stored only as salted hashes using a memory-hard function (e.g. Argon2/bcrypt). *Measure:* no reversible or plaintext credential appears in storage.

`NFR-SEC-4` **Transport security.**
All client–server traffic shall be over HTTPS/TLS in any deployed environment.

7.3 Performance, usability, and maintainability

`NFR-PERF-1` **Responsiveness.**
A check-in confirmation shall be processed within 500 ms at the 95th percentile under nominal load.

`NFR-PERF-2` **Prompt timeliness.**
A due check-in prompt shall be dispatched within 60 s of its scheduled time.

`NFR-USE-1` **One-action check-in.**
Confirming a check-in shall require exactly one deliberate user action and no re-authentication beyond the tokenised link (supports `FR-6`).

`NFR-MAINT-1` **Code quality.**
The codebase shall pass `ruff` with no errors and maintain unit-test coverage $\geq 80\%$ on the `crypto` and `scheduler` packages.

`NFR-PORT-1` **Storage portability.**
The system shall run against SQLite (development) and PostgreSQL (later) without code changes, via the SQLAlchemy abstraction.

8 Assumptions and Constraints

8.1 Assumptions

- The Owner and every trustee have a working, monitored email address.
- Trustees are cooperative and, after release, are willing to combine their shares to open the vault.
- The server's scheduling clock is the authoritative time source; the Owner's timezone is recorded for display only.

- Email delivery (SMTP) is reasonably reliable; transient failures are retried.

8.2 Constraints

- **Platform:** Python 3 + FastAPI backend, SQLAlchemy persistence, APScheduler scheduling, SMTP email, React + Tailwind frontend.
- **Algorithmic:** Shamir's Secret Sharing is hand-written (finite-field polynomial evaluation + Lagrange interpolation), not taken from a third-party secret-sharing library; no HSM is used in v1.
- **Product:** the v1 non-goals (NG-1…NG-3) hold — email only, web only, one vault per Owner.
- **Process:** the system is developed by a three-member team over a 12-week UCS503P timeline with weekly, CI-backed increments.

9 Requirement traceability *…forward references*

The functional and non-functional identifiers defined here are cited directly in the module documentation (`code/*/README.md`) and will be cited by the corresponding tests as they are written, giving a requirement → module → test trace. The vault lifecycle referenced by `FR-5`–`FR-7` and `NFR-REL-1` is specified by the state diagram in `docs/diagrams/vault-lifecycle.md`.